

3D Data Processing, Lab 3: Full Cloud Registration

Giuseppe D'Auria, ID 2163508

1 Overview

This report describes my implementation of full cloud registration on three datasets, bunny, dragon and vase. The pipeline is two stage. A descriptor based global alignment provides a coarse initial pose. An iterative closest point refinement then converges to a sub millimetre alignment on bunny and dragon, and to a millimetre level alignment on vase. I implemented the descriptor stage with FPFH features on a voxel downsampled support set, followed by mutual nearest neighbour matching in the 33 dimensional feature space and a RANSAC filter. I implemented two custom ICP variants, a closed form SVD solver and a non linear Levenberg Marquardt solver based on the Ceres library, and I compared them against the three reference Open3D solvers point to point, point to plane and generalised ICP under a 4×4 grid of injected rotation and translation noise.

2 Implementation

2.1 Closest point correspondence search: `find_closest_point`

For each point $\mathbf{p}_i$ of the current source cloud I query the target KD tree for its nearest neighbour $\mathbf{q}_j$. I accept the correspondence only if $\|\mathbf{p}_i - \mathbf{q}_j\|^2 < \tau^2$, where τ is the inlier threshold passed by the caller. I avoid the per point square root by comparing squared distances. I return the indices into the source and target clouds and the inlier root mean square error

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{I}|} \sum_{(i,j) \in \mathcal{I}} \|\mathbf{p}_i - \mathbf{q}_j\|^2}. \quad (1)$$

2.2 Closed form ICP step: `get_svd_icp_transformation`

Given a set of point correspondences I compute the centroids $\mathbf{c}_s$ and $\mathbf{c}_t$ of source and target inliers, centre both sets by subtracting the centroids, and form the cross covariance

$$\mathbf{H} = \sum_i (\mathbf{s}_i - \mathbf{c}_s)(\mathbf{t}_i - \mathbf{c}_t)^\top \in \mathbb{R}^{3 \times 3}. \quad (2)$$

I then run the Jacobi singular value decomposition $\mathbf{H} = \mathbf{U}\Sigma\mathbf{V}^\top$ through `Eigen::JacobiSVD<Eigen::MatrixXd>`, which is the type the assignment explicitly mandates. The rotation closes the orientation alignment with the reflection corrected form

$$\mathbf{R} = \mathbf{V} \text{diag}(1, 1, \det(\mathbf{V}\mathbf{U}^\top)) \mathbf{U}^\top. \quad (3)$$

The determinant factor is silently $+1$ when $\det(\mathbf{V}\mathbf{U}^\top) > 0$. When the bare $\mathbf{V}\mathbf{U}^\top$ would instead be an improper rotation, a rotation combined with a reflection, the factor flips the sign of the third column. The translation closes the centroid alignment, $\mathbf{t} = \mathbf{c}_t - \mathbf{R}\mathbf{c}_s$.

2.3 Non linear ICP step: `get_lm_icp_transformation`

I parameterise the rigid transformation with a six dimensional vector $(r_x, r_y, r_z, t_x, t_y, t_z)$, the first three components being the axis angle representation of the rotation. I build a Ceres problem with one residual block per correspondence, using the auto differentiated functor `PointDistance` with template arguments `<3, 6>`, where 3 is the residual dimension and 6 is the parameter block size. The residual is

$$\mathbf{r}_i = \mathbf{R}(\mathbf{r})\mathbf{s}_i + \mathbf{t} - \mathbf{t}_i. \quad (4)$$

The solver uses the Levenberg Marquardt trust region strategy with a dense QR linear solver, up to 50 iterations, and the initial parameter vector at zero, which represents the identity transformation. The output axis angle vector is converted to a rotation matrix through `ceres::AngleAxisToRotationMatrix`, which writes the result into a flat array in column major order, so I unpack it row by row when constructing the `Eigen::Matrix3d`.

2.4 Descriptor based global registration: `execute_descriptor_registration`

The initial alignment uses FPFH descriptors computed on the voxel downsampled support set, which is the canonical Open3D recipe for global registration. I made all hyperparameters adaptive with respect to the bounding box diagonal. Let

$$\ell = \max(\|\mathbf{b}_s^{\max} - \mathbf{b}_s^{\min}\|, \|\mathbf{b}_t^{\max} - \mathbf{b}_t^{\min}\|) \quad (5)$$

be the maximum diagonal of the two axis aligned bounding boxes. The pipeline parameters are

$$v = 0.02 \ell, \quad \text{voxel size for downsampling}, \quad (6)$$

$$r_n = 2v, \quad \text{radius for normal estimation}, \quad (7)$$

$$r_f = 5v, \quad \text{radius for FPFH feature extraction}, \quad (8)$$

$$d_R = 1.5v, \quad \text{RANSAC inlier distance}. \quad (9)$$

These are the ratios most Open3D tutorials use for FPFH based registration on millimetre scale meshes. Voxel downsampling drops the point count by roughly a factor of 100, which is what makes the cubic descriptor matching tractable on these clouds, and the regular spacing of the voxel grid also gives a stable support set for the descriptor stage.

I considered adding an explicit keypoint detection stage with Intrinsic Shape Signatures, but the default ISS configuration produced only a handful of keypoints on the smooth surfaces of the provided datasets, which left mutual nearest neighbour matching with one or two correspondences and made RANSAC fail. The canonical Open3D global registration recipe avoids this failure mode by treating every voxel downsampled point as a candidate, which is what I adopted.

After downsampling I estimate normals on the downsampled clouds with a hybrid KD tree search of radius r_n and at most 30 neighbours, and I normalise them. I then compute FPFH descriptors of dimension 33 through `open3d::pipelines::registration::ComputeFPFHFeature` with a hybrid KD tree search using r_f and at most 100 neighbours, applied to every downsampled point.

For the correspondence search I deliberately use mutual nearest neighbour matching rather than a one directional nearest neighbour. I build a `KDTreeFlann` on each of the two feature sets, find for each source feature its closest target feature, find for each target feature its closest source feature, and retain only the pairs (i, j) such that j is the nearest neighbour of i in the target and i is the nearest neighbour of j in the source. This filtering removes a large fraction of geometrically incompatible matches before they reach RANSAC.

The final transformation is estimated with `RegistrationRANSACBasedOnCorrespondence`, with a point to point estimator, a sample size of 3, an edge length checker with ratio 0.9, a distance checker at d_R , and a convergence criterion of 10^5 maximum iterations or 0.999 confidence. The result is written into `transformation_` and consumed by the subsequent ICP stage as the initial guess.

2.5 Outer ICP loop and the corrected starter code

The outer ICP loop is provided in `Registration.cpp`, in the function `execute_icp_registration`. Each iteration calls `find_closest_point`, runs either the SVD or LM step on the inlier set, applies the transformation to the working source cloud, and accumulates it into `transformation_`. The loop stops when the inlier RMSE drops below the relative threshold or hits the iteration cap.

The starter code as first distributed had two bugs in this function. The course has since released a corrected starter with both fixes, and this submission builds on that corrected version, so the loop is the official one unchanged. I had reached the same two corrections while debugging an anomalous post ICP RMSE before the corrected starter was released, and I summarise them here because the accumulation fix is what makes the RMSE numbers in Section 4, and the comparison between the custom and Open3D variants, valid.

The first was the composition order of the accumulator. The corrected loop left multiplies, `transformation_ = transformation * transformation_`, which is required because Open3D's `Transform` applies the matrix on the left of the column vector coordinates. The original right multiplication built a different product, so `compute_rmse` reapplied to the original source cloud a pose the loop had never actually reached. On bunny that inflated the final RMSE from 0.21 mm to 0.69 mm, so the refinement appeared to make the result worse than the descriptor stage alone, which is the symptom that led me to it. The second was the Open3D convergence criteria, originally written with Python style keyword arguments. C++ has no named arguments, so each `name = value` is parsed as an assignment expression and bound positionally, which silently swapped the RMSE tolerance with the iteration count and left the iteration limit at its default of 30. The corrected call passes the values positionally, `ICPConvergenceCriteria(1e-6, relative_rmse, max_iteration)`.

3 Experimental setup

3.1 Datasets and pipeline

The three datasets are bunny, dragon and vase, in millimetre units. For each dataset I run the descriptor stage once, then the ICP stage from the descriptor pose, and I report the resulting alignment under the metric `compute_rmse`, which is the full cloud point to nearest point root mean square error after applying the final pose to the original source cloud.

For the noise sweep I add a synthetic perturbation to the descriptor pose, with rotation noise standard deviation $\sigma_r \in \{0, 3, 5, 10\}$ degrees and translation noise standard deviation $\sigma_t \in \{0, 1, 3, 5\}$ millimetres, and I run the five ICP variants from this perturbed pose. The noise generator is seeded with the system time at startup, so the absolute values shift from run to run while the qualitative patterns are stable.

3.2 Inlier threshold scheduling

For the single noise free run I use the rule `threshold = 10.0`, which is large enough to absorb the descriptor stage residual of a few millimetres on every dataset, after which the ICP rapidly shrinks the inlier set as it converges.

For the noise sweep I use the rule

$$\tau = \max(1, 2(\sigma_t + \rho \sigma_r)) \quad \text{mm}, \quad (10)$$

where σ_t is the translation noise in millimetres, σ_r is the rotation noise in degrees, and $\rho = 1.5$ is an empirical tuning constant rather than a physical conversion factor. The 1 millimetre floor handles the zero noise case. The linear term in σ_t scales with the translation noise directly. The $\rho \sigma_r$ term is a coarse proxy for the surface displacement induced by a small rotation about the

centroid; the constant was calibrated empirically on bunny. As I show in Section 4, the 1 mm floor turns out to be too tight for the noise free vase row, which is the most informative single cell in the sweep.

4 Results

4.1 Descriptor stage metrics

Table 1 summarises the descriptor stage on the three datasets. The mutual correspondence count and RANSAC fitness reflect how well the FPFH features discriminate the surface of each cloud. On bunny and dragon the surfaces have enough local variation to produce more than a thousand mutual matches, RANSAC fitness reaches the maximum value of 1.0, and the resulting initial RMSE is sub millimetre. On vase the surface is smoother and largely rotationally symmetric on the body, which collapses many descriptors to similar histograms, so the mutual matching produces only 102 surviving pairs and the residual after RANSAC is larger.

Dataset	Mutual matches	RANSAC fitness	Inlier RMSE [mm]	Full cloud RMSE [mm]
bunny	1571	1.000	0.700	0.452
dragon	1084	1.000	0.940	0.614
vase	102	0.893	1.980	1.839

Table 1: Descriptor stage metrics for the three datasets.

4.2 Single mode ICP results

Running each dataset with the custom SVD mode from the descriptor pose, with $\tau = 10$ mm and a maximum of 100 iterations, yields:

- Bunny: 0.452 mm $\rightarrow$ 0.212 mm, 16 iterations, 235 ms.
- Dragon: 0.614 mm $\rightarrow$ 0.433 mm, 19 iterations, 152 ms.
- Vase: 1.839 mm $\rightarrow$ 1.180 mm, near max iterations, around 2.2 s.

The ICP delivers a $2.1\times$, $1.4\times$ and $1.6\times$ improvement over the descriptor stage on bunny, dragon and vase respectively. The vase needs a large fraction of the iteration budget because the surface symmetry leaves the cost function approximately flat along a rotation around the vase axis, which the loop traverses slowly.

These single mode values are slightly different from the noise free rows of Tables 2, 3 and 4 because the two experiments use different inlier thresholds: the single mode runs with $\tau = 10$ mm while the noise sweep at $\sigma_r = \sigma_t = 0$ runs with the 1 mm floor of the rule in Section 3.2. The two thresholds select different inlier subsets at iteration zero and therefore reach minima that differ slightly in the third decimal place.

4.3 Qualitative figures

Figures 1, 2 and 3 show, for each dataset, the source in orange and the target in blue at three pipeline stages: before any alignment, after the descriptor stage, and after the ICP refinement. The visual progression matches the numerical one. In the initial column the source and target are visibly separated. In the descriptor column the surfaces overlap with sub millimetre residual on bunny and dragon and a small residual on vase. In the ICP column the overlap is essentially complete.

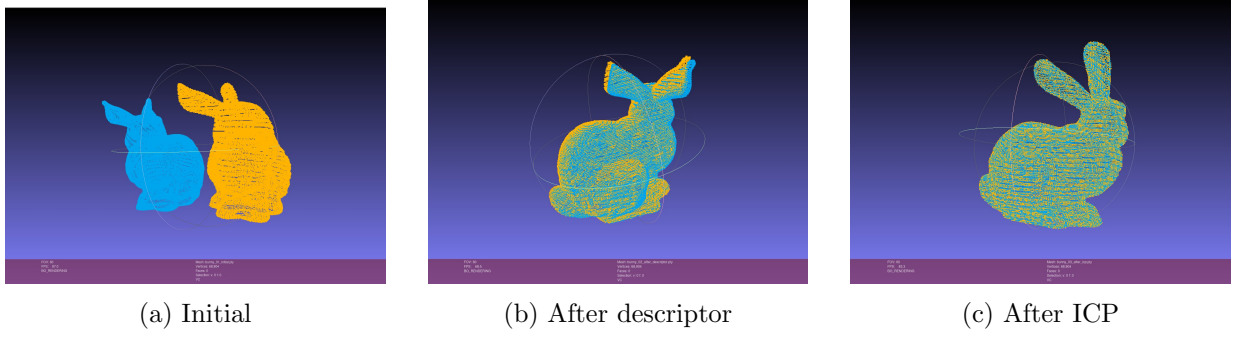


Figure 1: Bunny dataset, alignment progression. Source orange, target blue.

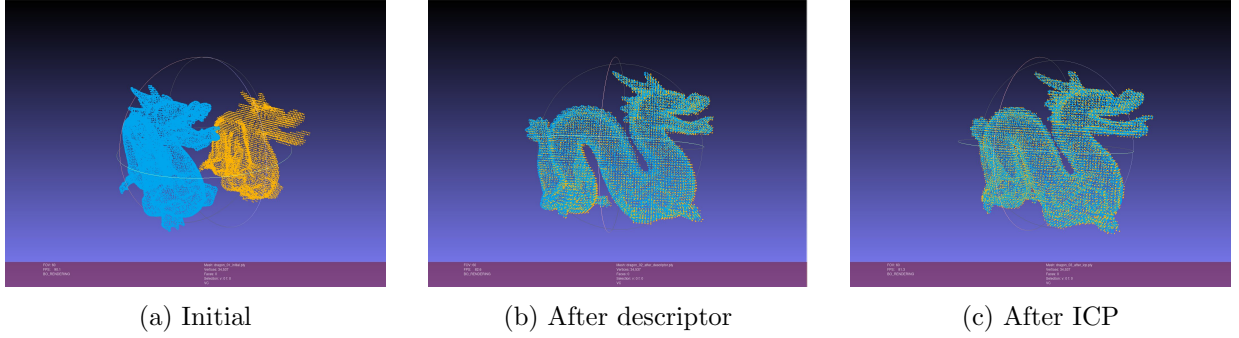


Figure 2: Dragon dataset, alignment progression. Source orange, target blue.

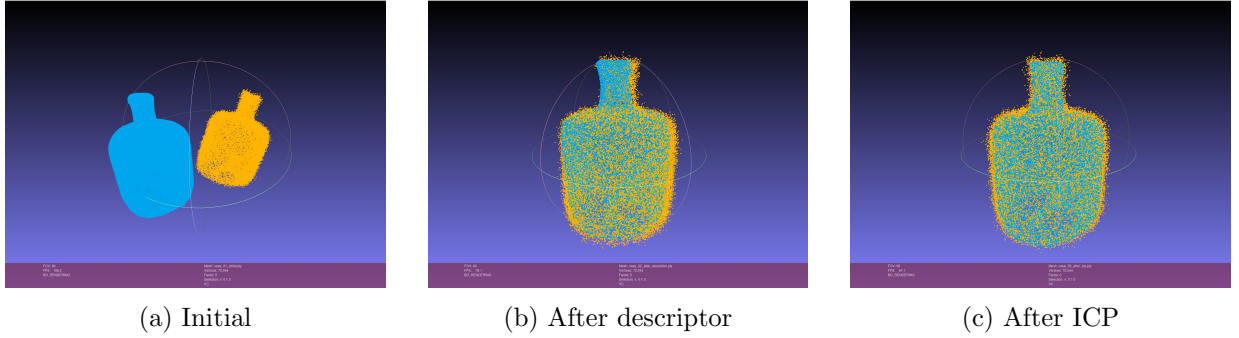


Figure 3: Vase dataset, alignment progression. Source orange, target blue.

4.4 Noise sweep tables

Tables 2, 3 and 4 report the full 4×4 noise grid for each of the five ICP variants on each dataset. Columns are rotation noise standard deviation in degrees, translation noise standard deviation in millimetres, ICP variant, number of iterations performed, wall clock time in milliseconds, and final RMSE in millimetres. The number of iterations for the three Open3D variants is reported as 0 because the Open3D `RegistrationICP` entry point does not return that field; the actual count can be recovered from the debug stream.

Table 2: Bunny dataset, full noise grid for all five ICP variants.

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
0.000	0.000	svd	15	228.610	0.212
0.000	0.000	lm	15	907.037	0.212
0.000	0.000	o3d-p2point	0	43.458	0.212
0.000	0.000	o3d-p2plane	0	63.744	0.212

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
0.000	0.000	o3d-gen	0	111.724	0.212
0.000	1.000	svd	16	241.829	0.212
0.000	1.000	lm	16	910.684	0.212
0.000	1.000	o3d-p2point	0	42.841	0.212
0.000	1.000	o3d-p2plane	0	63.201	0.212
0.000	1.000	o3d-gen	0	98.003	0.212
0.000	3.000	svd	18	287.843	0.212
0.000	3.000	lm	18	1008.273	0.212
0.000	3.000	o3d-p2point	0	50.191	0.212
0.000	3.000	o3d-p2plane	0	63.375	0.212
0.000	3.000	o3d-gen	0	112.999	0.212
0.000	5.000	svd	20	338.209	0.212
0.000	5.000	lm	20	1140.665	0.212
0.000	5.000	o3d-p2point	0	58.913	0.212
0.000	5.000	o3d-p2plane	0	75.469	0.212
0.000	5.000	o3d-gen	0	101.576	0.212
3.000	0.000	svd	16	235.632	0.212
3.000	0.000	lm	16	932.637	0.212
3.000	0.000	o3d-p2point	0	44.729	0.212
3.000	0.000	o3d-p2plane	0	16.282	0.212
3.000	0.000	o3d-gen	0	115.166	0.212
3.000	1.000	svd	27	425.233	0.212
3.000	1.000	lm	27	1677.442	0.212
3.000	1.000	o3d-p2point	0	176.633	0.212
3.000	1.000	o3d-p2plane	0	37.244	0.212
3.000	1.000	o3d-gen	0	116.363	0.212
3.000	3.000	svd	21	334.113	0.212
3.000	3.000	lm	21	1239.045	0.212
3.000	3.000	o3d-p2point	0	63.169	0.212
3.000	3.000	o3d-p2plane	0	20.174	0.212
3.000	3.000	o3d-gen	0	101.228	0.212
3.000	5.000	svd	22	364.170	0.212
3.000	5.000	lm	22	1374.526	0.212
3.000	5.000	o3d-p2point	0	63.825	0.212
3.000	5.000	o3d-p2plane	0	78.650	0.212
3.000	5.000	o3d-gen	0	89.104	0.212
5.000	0.000	svd	26	400.618	0.212
5.000	0.000	lm	26	1648.326	0.212
5.000	0.000	o3d-p2point	0	66.909	0.212
5.000	0.000	o3d-p2plane	0	74.800	0.212
5.000	0.000	o3d-gen	0	115.332	0.212
5.000	1.000	svd	31	514.734	0.212
5.000	1.000	lm	31	2027.681	0.212
5.000	1.000	o3d-p2point	0	81.477	0.212
5.000	1.000	o3d-p2plane	0	77.636	0.212
5.000	1.000	o3d-gen	0	79.559	0.212
5.000	3.000	svd	23	376.556	0.212
5.000	3.000	lm	23	1449.960	0.212
5.000	3.000	o3d-p2point	0	60.662	0.212
5.000	3.000	o3d-p2plane	0	67.514	0.212

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
5.000	3.000	o3d-gen	0	111.370	0.212
5.000	5.000	svd	24	411.328	0.212
5.000	5.000	lm	24	1408.726	0.212
5.000	5.000	o3d-p2point	0	72.526	0.212
5.000	5.000	o3d-p2plane	0	21.724	0.212
5.000	5.000	o3d-gen	0	124.176	0.212
10.000	0.000	svd	27	475.274	0.212
10.000	0.000	lm	27	1734.491	0.212
10.000	0.000	o3d-p2point	0	82.413	0.212
10.000	0.000	o3d-p2plane	0	92.344	0.212
10.000	0.000	o3d-gen	0	114.554	0.212
10.000	1.000	svd	26	434.190	0.212
10.000	1.000	lm	26	1616.817	0.212
10.000	1.000	o3d-p2point	0	79.322	0.212
10.000	1.000	o3d-p2plane	0	26.276	0.212
10.000	1.000	o3d-gen	0	136.477	0.212
10.000	3.000	svd	29	493.320	0.212
10.000	3.000	lm	29	1861.817	0.212
10.000	3.000	o3d-p2point	0	77.219	0.212
10.000	3.000	o3d-p2plane	0	24.686	0.212
10.000	3.000	o3d-gen	0	127.245	0.212
10.000	5.000	svd	27	460.786	0.212
10.000	5.000	lm	27	1679.923	0.212
10.000	5.000	o3d-p2point	0	81.994	0.212
10.000	5.000	o3d-p2plane	0	21.399	0.212
10.000	5.000	o3d-gen	0	123.380	0.212

Table 3: Dragon dataset, full noise grid for all five ICP variants.

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
0.000	0.000	svd	21	161.359	0.433
0.000	0.000	lm	21	458.396	0.433
0.000	0.000	o3d-p2point	0	22.144	0.433
0.000	0.000	o3d-p2plane	0	10.787	0.433
0.000	0.000	o3d-gen	0	34.280	0.433
0.000	1.000	svd	22	168.570	0.433
0.000	1.000	lm	22	491.676	0.433
0.000	1.000	o3d-p2point	0	21.143	0.433
0.000	1.000	o3d-p2plane	0	11.818	0.433
0.000	1.000	o3d-gen	0	97.606	0.433
0.000	3.000	svd	23	182.636	0.433
0.000	3.000	lm	23	478.180	0.433
0.000	3.000	o3d-p2point	0	24.075	0.433
0.000	3.000	o3d-p2plane	0	11.474	0.433
0.000	3.000	o3d-gen	0	48.338	0.433
0.000	5.000	svd	25	202.306	0.433
0.000	5.000	lm	25	532.326	0.433
0.000	5.000	o3d-p2point	0	25.778	0.433

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
0.000	5.000	o3d-p2plane	0	16.443	0.433
0.000	5.000	o3d-gen	0	67.167	0.433
3.000	0.000	svd	23	176.810	0.433
3.000	0.000	lm	23	501.589	0.433
3.000	0.000	o3d-p2point	0	23.859	0.433
3.000	0.000	o3d-p2plane	0	10.301	0.433
3.000	0.000	o3d-gen	0	43.522	0.433
3.000	1.000	svd	26	201.322	0.433
3.000	1.000	lm	27	588.446	0.433
3.000	1.000	o3d-p2point	0	27.011	0.433
3.000	1.000	o3d-p2plane	0	12.253	0.433
3.000	1.000	o3d-gen	0	46.641	0.433
3.000	3.000	svd	25	197.375	0.433
3.000	3.000	lm	24	531.834	0.433
3.000	3.000	o3d-p2point	0	24.820	0.433
3.000	3.000	o3d-p2plane	0	9.605	0.433
3.000	3.000	o3d-gen	0	34.020	0.433
3.000	5.000	svd	27	218.376	0.433
3.000	5.000	lm	27	572.983	0.433
3.000	5.000	o3d-p2point	0	26.670	0.433
3.000	5.000	o3d-p2plane	0	9.863	0.433
3.000	5.000	o3d-gen	0	41.437	0.433
5.000	0.000	svd	26	201.369	0.433
5.000	0.000	lm	26	559.242	0.433
5.000	0.000	o3d-p2point	0	23.239	0.433
5.000	0.000	o3d-p2plane	0	8.716	0.433
5.000	0.000	o3d-gen	0	37.890	0.433
5.000	1.000	svd	26	210.734	0.433
5.000	1.000	lm	26	575.027	0.433
5.000	1.000	o3d-p2point	0	55.352	0.433
5.000	1.000	o3d-p2plane	0	10.135	0.433
5.000	1.000	o3d-gen	0	36.920	0.433
5.000	3.000	svd	26	208.031	0.433
5.000	3.000	lm	26	575.882	0.433
5.000	3.000	o3d-p2point	0	79.782	0.433
5.000	3.000	o3d-p2plane	0	102.861	0.433
5.000	3.000	o3d-gen	0	37.370	0.433
5.000	5.000	svd	35	289.743	0.433
5.000	5.000	lm	35	756.029	0.433
5.000	5.000	o3d-p2point	0	34.577	0.433
5.000	5.000	o3d-p2plane	0	106.542	0.433
5.000	5.000	o3d-gen	0	62.757	0.433
10.000	0.000	svd	24	191.318	0.433
10.000	0.000	lm	24	539.339	0.433
10.000	0.000	o3d-p2point	0	22.929	0.433
10.000	0.000	o3d-p2plane	0	17.186	0.433
10.000	0.000	o3d-gen	0	123.755	0.433
10.000	1.000	svd	38	308.178	0.433
10.000	1.000	lm	38	881.541	0.433
10.000	1.000	o3d-p2point	0	38.857	0.433

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
10.000	1.000	o3d-p2plane	0	120.281	0.433
10.000	1.000	o3d-gen	0	39.047	0.433
10.000	3.000	svd	29	239.339	0.433
10.000	3.000	lm	29	654.071	0.433
10.000	3.000	o3d-p2point	0	32.317	0.433
10.000	3.000	o3d-p2plane	0	12.551	0.433
10.000	3.000	o3d-gen	0	141.520	0.433
10.000	5.000	svd	29	235.463	0.433
10.000	5.000	lm	29	653.480	0.433
10.000	5.000	o3d-p2point	0	48.373	0.433
10.000	5.000	o3d-p2plane	0	11.858	0.433
10.000	5.000	o3d-gen	0	43.165	0.433

Table 4: Vase dataset, full noise grid for all five ICP variants.

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
0.000	0.000	svd	3	99.278	1.739
0.000	0.000	lm	3	144.543	1.739
0.000	0.000	o3d-p2point	0	311.384	1.174
0.000	0.000	o3d-p2plane	0	94.117	1.169
0.000	0.000	o3d-gen	0	1370.455	1.212
0.000	1.000	svd	28	652.624	1.176
0.000	1.000	lm	30	1797.881	1.175
0.000	1.000	o3d-p2point	0	425.311	1.171
0.000	1.000	o3d-p2plane	0	213.041	1.168
0.000	1.000	o3d-gen	0	763.058	1.171
0.000	3.000	svd	100	2166.709	1.171
0.000	3.000	lm	100	6401.694	1.171
0.000	3.000	o3d-p2point	0	363.835	1.171
0.000	3.000	o3d-p2plane	0	86.919	1.168
0.000	3.000	o3d-gen	0	484.097	1.169
0.000	5.000	svd	100	2195.945	1.171
0.000	5.000	lm	100	6383.923	1.171
0.000	5.000	o3d-p2point	0	362.872	1.171
0.000	5.000	o3d-p2plane	0	115.029	1.168
0.000	5.000	o3d-gen	0	1205.874	1.169
3.000	0.000	svd	100	2184.681	1.182
3.000	0.000	lm	100	6332.684	1.182
3.000	0.000	o3d-p2point	0	350.090	1.182
3.000	0.000	o3d-p2plane	0	133.287	1.168
3.000	0.000	o3d-gen	0	824.428	1.169
3.000	1.000	svd	100	2180.404	1.178
3.000	1.000	lm	100	6211.227	1.178
3.000	1.000	o3d-p2point	0	418.899	1.178
3.000	1.000	o3d-p2plane	0	146.979	1.168
3.000	1.000	o3d-gen	0	1110.735	1.169
3.000	3.000	svd	89	1938.320	1.170
3.000	3.000	lm	89	5543.848	1.170

σ_r [deg]	σ_t [mm]	Method	Iters	Time [ms]	RMSE [mm]
3.000	3.000	o3d-p2point	0	307.087	1.170
3.000	3.000	o3d-p2plane	0	112.983	1.168
3.000	3.000	o3d-gen	0	332.869	1.169
3.000	5.000	svd	100	2186.797	1.176
3.000	5.000	lm	100	6207.156	1.176
3.000	5.000	o3d-p2point	0	344.079	1.176
3.000	5.000	o3d-p2plane	0	256.302	1.168
3.000	5.000	o3d-gen	0	1089.066	1.169
5.000	0.000	svd	79	1751.628	1.168
5.000	0.000	lm	65	4165.331	1.168
5.000	0.000	o3d-p2point	0	287.156	1.168
5.000	0.000	o3d-p2plane	0	106.699	1.168
5.000	0.000	o3d-gen	0	284.791	1.169
5.000	1.000	svd	100	2275.893	1.174
5.000	1.000	lm	100	6644.008	1.174
5.000	1.000	o3d-p2point	0	746.250	1.174
5.000	1.000	o3d-p2plane	0	138.534	1.168
5.000	1.000	o3d-gen	0	2085.822	1.169
5.000	3.000	svd	100	2710.330	1.169
5.000	3.000	lm	100	8178.962	1.169
5.000	3.000	o3d-p2point	0	865.800	1.169
5.000	3.000	o3d-p2plane	0	134.743	1.168
5.000	3.000	o3d-gen	0	781.476	1.170
5.000	5.000	svd	100	2573.000	1.176
5.000	5.000	lm	100	6947.180	1.176
5.000	5.000	o3d-p2point	0	390.566	1.176
5.000	5.000	o3d-p2plane	0	49.022	1.168
5.000	5.000	o3d-gen	0	908.149	1.169
10.000	0.000	svd	34	788.145	1.169
10.000	0.000	lm	35	2220.585	1.169
10.000	0.000	o3d-p2point	0	179.714	1.169
10.000	0.000	o3d-p2plane	0	131.434	1.168
10.000	0.000	o3d-gen	0	753.418	1.169
10.000	1.000	svd	100	2410.697	1.174
10.000	1.000	lm	100	6608.488	1.174
10.000	1.000	o3d-p2point	0	377.333	1.174
10.000	1.000	o3d-p2plane	0	236.450	1.168
10.000	1.000	o3d-gen	0	808.010	1.169
10.000	3.000	svd	100	2245.264	1.179
10.000	3.000	lm	100	6437.601	1.179
10.000	3.000	o3d-p2point	0	316.228	1.179
10.000	3.000	o3d-p2plane	0	52.804	1.168
10.000	3.000	o3d-gen	0	1386.441	1.169
10.000	5.000	svd	73	1705.753	1.168
10.000	5.000	lm	74	4640.997	1.168
10.000	5.000	o3d-p2point	0	302.681	1.168
10.000	5.000	o3d-p2plane	0	178.143	1.168
10.000	5.000	o3d-gen	0	578.870	1.169

4.5 Discussion

On bunny and dragon all five variants reach the same minimum at zero noise. This is what I expected from a stable local minimum, the iteration recovers the same pose regardless of where the perturbation places the initial guess. The constancy of the RMSE column at 0.212 mm on bunny and 0.433 mm on dragon across the whole noise grid is what this looks like in the table: ICP falls back into the same basin of attraction every time.

On timing the Open3D variants are much faster than my custom svd and lm. On bunny at zero noise the custom svd takes 229 ms and lm takes 907 ms, while o3d-p2point is at 44 ms, o3d-p2plane at 64 ms and o3d-gen at 112 ms. On dragon the gap is even larger, with svd at 161 ms versus an 11 ms o3d-p2plane. The reason is not algorithmic. My `find_closest_point` rebuilds the target `KDTreeFlann` every iteration, which is the dominant cost when the target has tens of thousands of points. Open3D builds the tree once outside the loop and reuses it, plus it parallelises the per correspondence work across threads. Moving the KD tree to a class member would close most of the gap. I left it as is for this submission because the fix changes the function signature and the assignment uses the function as written. Generalised ICP is the slowest of the three Open3D variants on bunny because the local covariance computation adds work each iteration, but it is still about twice as fast as my custom svd.

On vase the picture is different. The vase is almost rotationally symmetric, so the cost function barely changes when you rotate the source around the vertical axis. The custom svd and lm have nothing to optimise along that direction and traverse it slowly, hitting the 100 iteration cap in many cells. Point to plane handles this better because it only penalises motion normal to the surface, not motion along it. It converges to a slightly smaller final RMSE, by 0.005 to 0.01 mm, in tens to low hundreds of milliseconds, and the pattern is consistent across all 16 cells. Generalised ICP tracks point to plane on vase with a small overhead from the covariance.

The strangest cell in any of the three tables is the noise free vase row. The custom svd at $\sigma_r = 0$, $\sigma_t = 0$ returns RMSE 1.739 mm after only 3 iterations and 99 ms. The same cell at $\sigma_t = 1$ mm returns 1.176 mm after 28 iterations and 652 ms. Adding 1 mm of translation noise made the result better by more than half a millimetre, which is the opposite of what you would expect. I traced it back to the threshold rule of Section 3.2. At zero noise the rule clamps τ to the 1 mm floor, which is below the descriptor stage RMSE of 1.84 mm on vase. The inlier set is then filtered to only the easiest points, the loop converges to a poor local minimum in 3 iterations and exits. Any non zero translation noise widens τ enough to admit most of the surface, the loop runs many iterations, and the RMSE improves. The threshold rule should scale with the descriptor stage RMSE, not only with the injected noise; this is the obvious fix.

Translation noise is otherwise tolerated up to roughly the inter point spacing of the original meshes on all five variants. Past that, the initial inlier set excludes most of the surface and ICP cannot recover. The threshold rule keeps the inlier set non empty up to the largest grid cell, but the recovered alignment degrades smoothly as the noise grows.

5 Conclusion

I implemented the four required methods of the lab. The closed form SVD step and the non linear Levenberg Marquardt step are mathematically equivalent on the same correspondence set and produce the same minima on the noise free benchmark. SVD is faster than LM by between $1.5\times$ and $4\times$ depending on the dataset, in my implementation, though both are slower than the Open3D variants because I rebuild the target KD tree at every iteration. The FPFH based global alignment is essential for the pipeline to converge to a good local minimum on bunny and dragon, and it is enough to bring vase within the basin of attraction of point to plane ICP, which is the variant that handles the vase symmetry best. The starter code originally had two bugs in the ICP loop, the accumulator composition order and the convergence criteria call, which the course has since corrected in an updated starter; the accumulation bug in particular made the RMSE

numbers noticeably worse before it was fixed.